

[illegible]

CPU Address Bus Buffering

M2 Clock Buffering

CPU Address Bus Nybble Decoding

/ROMSEL, Register Access

CPU R/W, Register Read/Write

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:		Rev:
Size: USLdger	Date:	
KiCad E.D.A. 9.0.7		Id: 2/26

CPU Address Bus Buffering

M2 Clock Buffering

CPU Address Bus Nybble Decoding

CPU R/W, Register Read/Write

/ROMSEL, Register Access

Note: The high nybbles are not taking /ROMSEL or A15 into account, so they are 32kB mirrored at this point. "Reg_Access" is included in downstream logic to specify which 32kB region.

Issues seeing traces in this area.

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

CPU Address Bus Buffering

M2 Clock Buffering

CPU Address Bus Nybble Decoding

CPU R/W, Register Read/Write

/ROMSEL, Register Access

Note: The high nybbles are not taking /ROMSEL or A15 into account, so they are 32kB mirrored at this point. "Reg_Access" is included in downstream logic to specify which 32kB region.

Issues seeing traces in this area.

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

CPU Address Bus Buffering

M2 Clock Buffering

CPU Address Bus Nybble Decoding

CPU R/W, Register Read/Write

/ROMSEL, Register Access

Note: The high nybbles are not taking /ROMSEL or A15 into account, so they are 32kB mirrored at this point. "Reg_Access" is included in downstream logic to specify which 32kB region.

Issues seeing traces in this area.

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

CPU Address Bus Buffering

M2 Clock Buffering

CPU Address Bus Nybble Decoding

CPU R/W, Register Read/Write

/ROMSEL, Register Access

Note: The high nybbles are not taking /ROMSEL or A15 into account, so they are 32kB mirrored at this point. "Reg_Access" is included in downstream logic to specify which 32kB region.

Issues seeing traces in this area.

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

CPU Address Bus Buffering

M2 Clock Buffering

CPU Address Bus Nybble Decoding

CPU R/W, Register Read/Write

/ROMSEL, Register Access

Note: The high nybbles are not taking /ROMSEL or A15 into account, so they are 32kB mirrored at this point. "Reg_Access" is included in downstream logic to specify which 32kB region.

Issues seeing traces in this area.

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

CPU Address Bus Buffering

M2 Clock Buffering

CPU Address Bus Nybble Decoding

CPU R/W, Register Read/Write

/ROMSEL, Register Access

Pin	Signal	Pin	Signal	Pin	Signal	Pin	Signal
1	CPU_A0_Pin	17	CPU_A12_Pin	33	CPU_A10_Pin	49	CPU_A14_Pin
2	CPU_A1_Pin	18	CPU_A13_Pin	34	CPU_A11_Pin	50	
3	CPU_A2_Pin	19		35		51	
4	CPU_A3_Pin	20		36		52	
5	CPU_A4_Pin	21		37		53	
6	CPU_A5_Pin	22		38		54	
7	CPU_A6_Pin	23		39		55	
8	CPU_A7_Pin	24		40		56	
9	CPU_A8_Pin	25		41		57	
10	CPU_A9_Pin	26		42		58	
11	CPU_A10_Pin	27		43		59	
12	CPU_A11_Pin	28		44		60	
13	CPU_A12_Pin	29		45		61	
14	CPU_A13_Pin	30		46		62	
15		31		47		63	
16		32		48		64	

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

CPU Address Bus Buffering

M2 Clock Buffering

CPU Address Bus Nybble Decoding

CPU R/W, Register Read/Write

/ROMSEL, Register Access

Note: The high nybbles are not taking /ROMSEL or A15 into account, so they are 32kB mirrored at this point. "Reg_Access" is included in downstream logic to specify which 32kB region.

Issues seeing traces in this area.

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

CPU Address Bus Buffering

M2 Clock Buffering

CPU Address Bus Nybble Decoding

CPU R/W, Register Read/Write

/ROMSEL, Register Access

Note: The high nybbles are not taking /ROMSEL or A15 into account, so they are 32kB mirrored at this point. "Reg_Access" is included in downstream logic to specify which 32kB region.

Issues seeing traces in this area.

Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

CPU Address Bus Buffering

M2 Clock Buffering

CPU Address Bus Nybble Decoding

CPU R/W, Register Read/Write

/ROMSEL, Register Access

Note: The high nybbles are not taking /ROMSEL or A15 into account, so they are 32kB mirrored at this point. "Reg_Access" is included in downstream logic to specify which 32kB region.

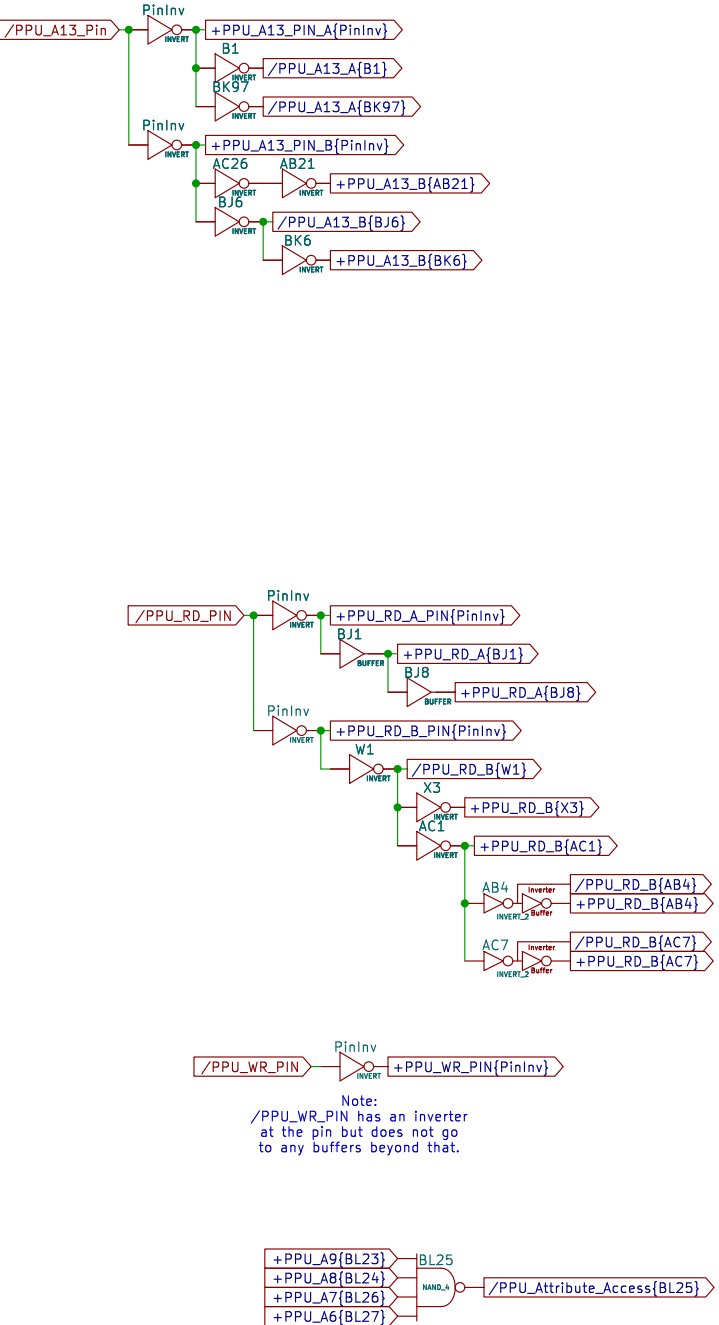
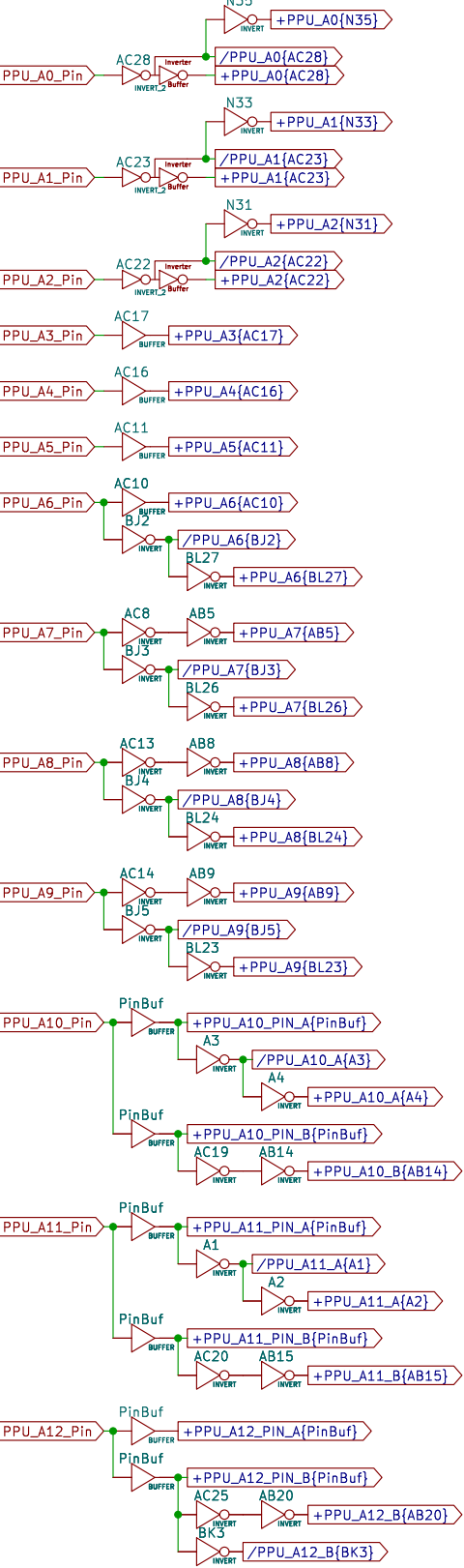
Issues seeing traces in this area.

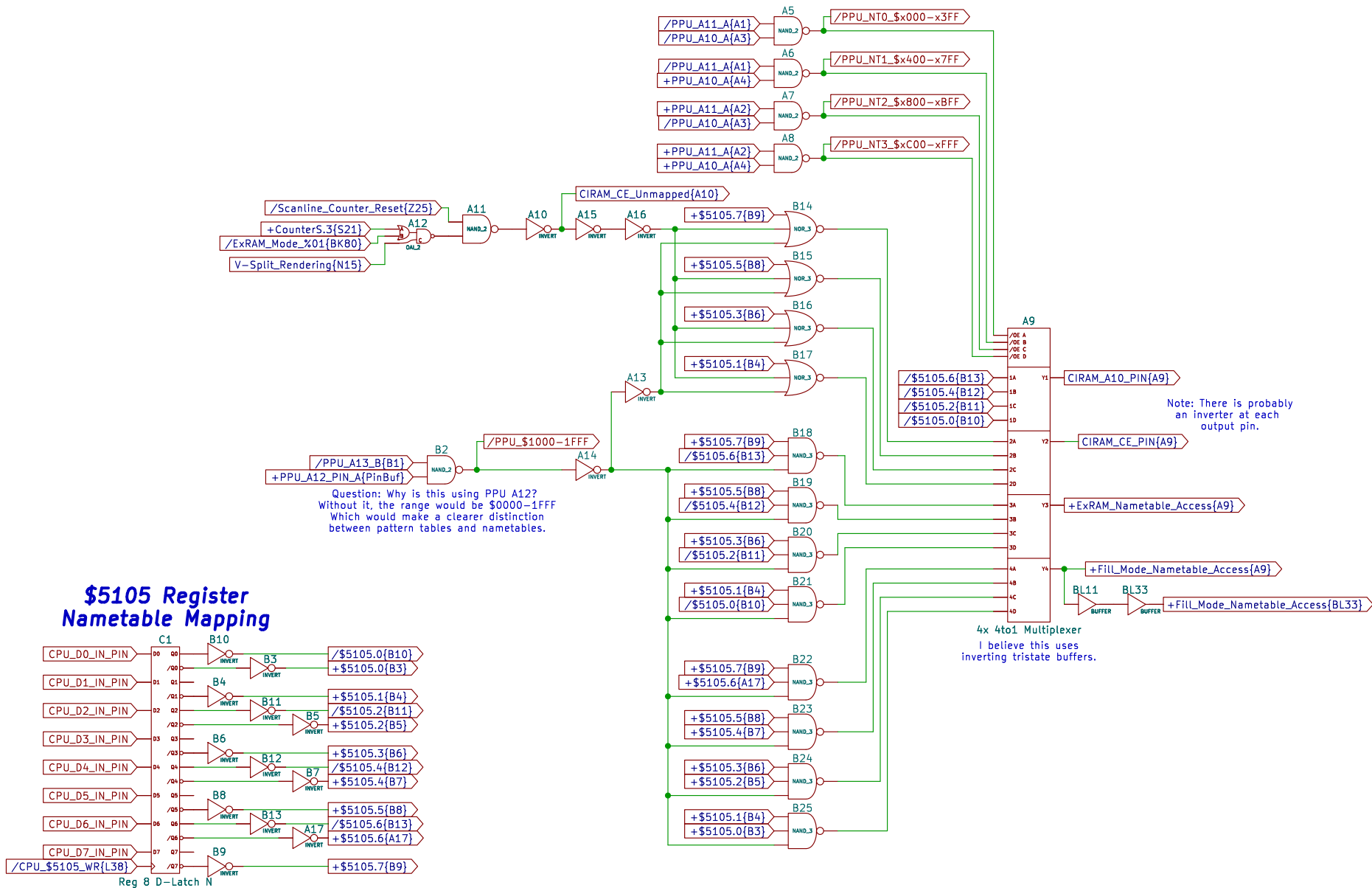
Sheet: /CPU Address Bus/
File: cpuAddressBus.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 2/26

PPU Address Bus Buffering





\$5100 Register PRG Mode

PRG Bank Registers \$5113 Register

\$5114 Register

\$5115 Register

\$5116 Register

\$5117 Register

\$5102 Register PRG-RAM Write Protection

\$5103 Register

Note: There is probably an inverter at each output pin.

Note: /Set vs. /Reset inputs are individually used but not easily shown here with a symbol.

Note: Q becomes locked asynchronously, becomes unlocked synchronized to M2

Sheet: /PRG Banking/
File: prg_banking.kicad_sch

Title:

Size: USLdger Date: Rev: KiCad E.D.A. 9.0.7 Id: 6/26

\$5100 Register PRG Mode

Inputs: +CPU_D0{D2}, +CPU_D1{D3}, /Reset_Detect{D1}, /CPU_\$5100_WR{L28}

Outputs: /\$5100.0{D9}, +\$5100.0{D10}, /\$5100.1{D11}, +\$5100.1{D12}

Reg 2 DFF-Set

PRG Bank Registers \$5113 Register

Inputs: /CPU_D0{E3}, /CPU_D1{E4}, /CPU_D2{E1}, /CPU_D3{E2}, /CPU_\$5113_WR{L46}

Outputs: /\$5113.0{B41}, /\$5113.1{B41}, /\$5113.2{B41}, /\$5113.3{B41}

Reg 4 D-Latch

\$5114 Register

Inputs: /CPU_D0{E3}, /CPU_D1{E4}, /CPU_D2{E1}, /CPU_D3{E2}, /CPU_D4{E5}, /CPU_D5{E7}, /CPU_D6{E6}, /CPU_D7{E8}, /CPU_\$5114_WR{L47}

Outputs: /\$5114.0{B40}, /\$5114.1{B40}, /\$5114.2{B40}, /\$5114.3{B40}, /\$5114.4{B40}, /\$5114.5{B40}, /\$5114.6{B40}, /\$5114.7{B40}

Reg 8 D-Latch

\$5115 Register

Inputs: /CPU_D0{E3}, /CPU_D1{E4}, /CPU_D2{E1}, /CPU_D3{E2}, /CPU_D4{E5}, /CPU_D5{E7}, /CPU_D6{E6}, /CPU_D7{E8}, /CPU_\$5115_WR{L49}

Outputs: /\$5115.0{B39}, /\$5115.1{B39}, /\$5115.2{B39}, /\$5115.3{B39}, /\$5115.4{B39}, /\$5115.5{B39}, /\$5115.6{B39}, /\$5115.7{B39}

Reg 8 D-Latch

\$5116 Register

Inputs: /CPU_D0{E3}, /CPU_D1{E4}, /CPU_D2{E1}, /CPU_D3{E2}, /CPU_D4{E5}, /CPU_D5{E7}, /CPU_D6{E6}, /CPU_D7{E8}, /CPU_\$5116_WR{L52}

Outputs: /\$5116.0{C7}, /\$5116.1{C7}, /\$5116.2{C7}, /\$5116.3{C7}, /\$5116.4{C7}, /\$5116.5{C7}, /\$5116.6{C7}, /\$5116.7{C7}

Reg 8 D-Latch

\$5117 Register

Inputs: /CPU_D0{E3}, /CPU_D1{E4}, /CPU_D2{E1}, /CPU_D3{E2}, /CPU_D4{E5}, /CPU_D5{E7}, /CPU_D6{E6}, /CPU_D7{E8}, +Reset_Detect{E13}, /CPU_\$5117_WR{L54}

Outputs: /\$5117.0{C6}, /\$5117.1{C6}, /\$5117.2{C6}, /\$5117.3{C6}, /\$5117.4{C6}, /\$5117.5{C6}, /\$5117.6{C6}, /\$5117.7{C6}

Reg 8 DFF-Set

Note: \$5117.7 RAM/ROM bit is fully implemented but is somehow being prevented from entering RAM mode when bench testing.

\$5102 Register PRG-RAM Write Protection

Inputs: +CPU_D0{E16}, +CPU_D1{E17}, /Reset_Detect{E18}, /CPU_\$5102_WR{L32}

Outputs: /\$5102.0{E19}, +\$5102.1{E19}

Reg 2 DFF-Reset N

\$5103 Register

Inputs: +CPU_D0{E16}, +CPU_D1{E17}, /Reset_Detect{E18}, /CPU_\$5103_WR{L35}

Outputs: +\$5103.0{E20}, /\$5103.1{E20}

Reg 2 DFF-Reset N

Note: /Set vs. /Reset inputs are individually used but not easily shown here with a symbol.

PRG-RAM Write Protection Logic

Inputs: +M2_CLK_BUF3{A38}, /ROMSEL_PIN, +RAM/ROM, +CPU_A14{A40}, +CPU_A13{A39}, /Reset_Detect{RAX12}, PRG_RAM_A15_Prebuf

Outputs: /PRG_ROM_CE_PIN, /PRG_RAM_CE_PIN, /PRG_RAM_CE_1_PIN, /PRG_RAM_CE_0_PIN, PRG_RAM_A15_PIN

PRG Bank Logic

Inputs: /\$5117.1{C6}, /\$5117.0{C6}, /CPU_A14{F2}, /CPU_A13{F1}, /CPU_A13{F1}, /CPU_A13{F1}

Outputs: Banks A14 for 8k & 16k banks. PRG_ROM_A14_PIN, Banks A13 for 8k banks. PRG_ROM_A13_PIN, Passes through A14 for 32k banks. PRG_ROM_A14_PIN, Passes through A14 for 32k banks. PRG_ROM_A13_PIN, Passes through A13 for 16k banks. PRG_ROM_A13_PIN

Note: There is probably an inverter at each output pin.

PRG-RAM Write Protection Logic (Continued)

Inputs: +M2_CLK_BUF3{E25}, /Reset /Q, +M2_CLK_BUF3{E14}, /M2_CLK_BUF3{E15}, /ROMSEL{E23,E24}, /CPU_R/W{E27}

Outputs: PRG-RAM_WE_PIN{E30}

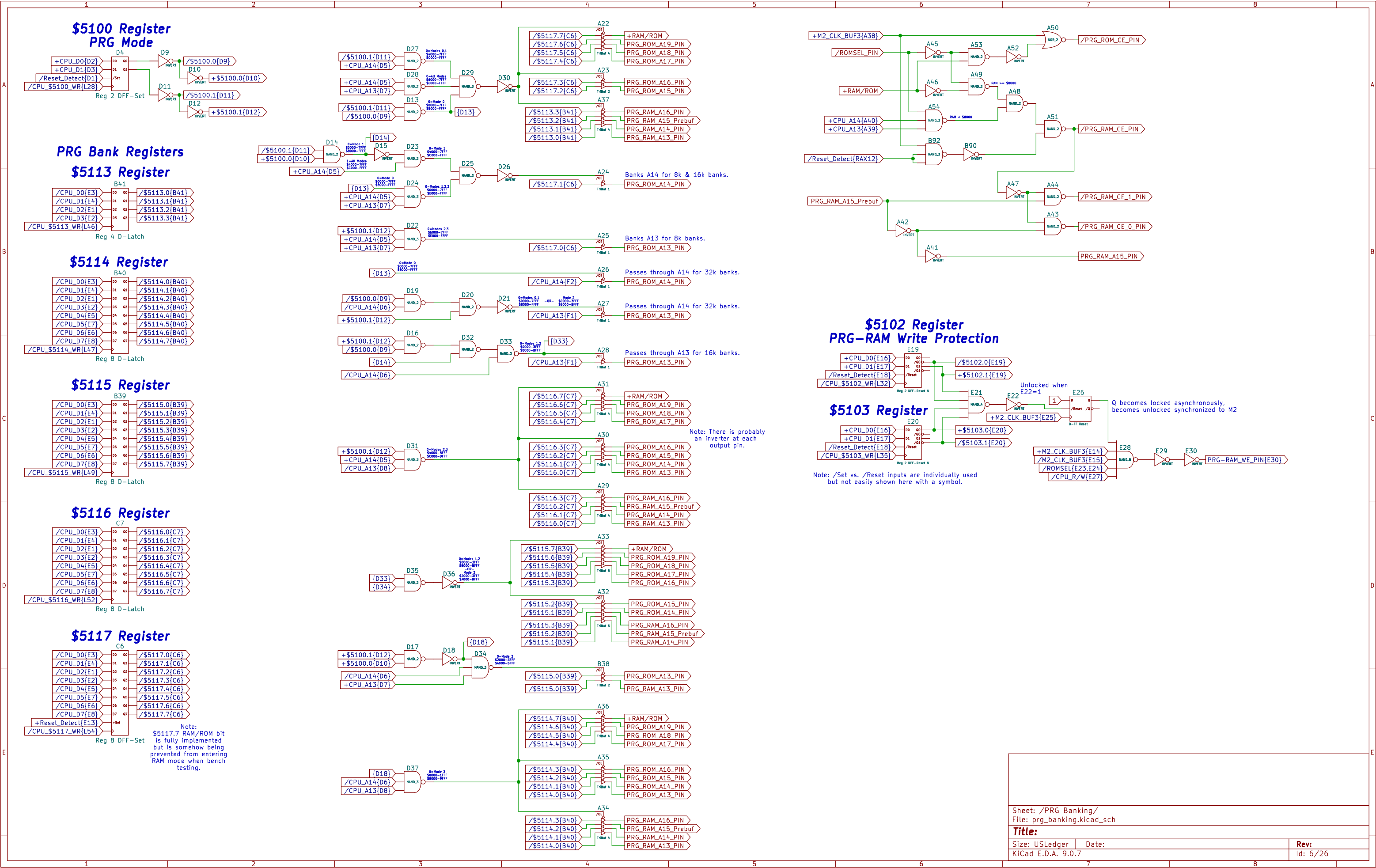
Unlocked when E22=1

Q becomes locked asynchronously, becomes unlocked synchronized to M2

Sheet: /PRG Banking/
File: prg_banking.kicad_sch

Title:

Size: USLdger Date: Rev: KiCad E.D.A. 9.0.7 Id: 6/26



\$5100 Register PRG Mode

PRG Bank Registers \$5113 Register

\$5114 Register

\$5115 Register

\$5116 Register

\$5117 Register

\$5102 Register PRG-RAM Write Protection

\$5103 Register

Note: There is probably an inverter at each output pin.

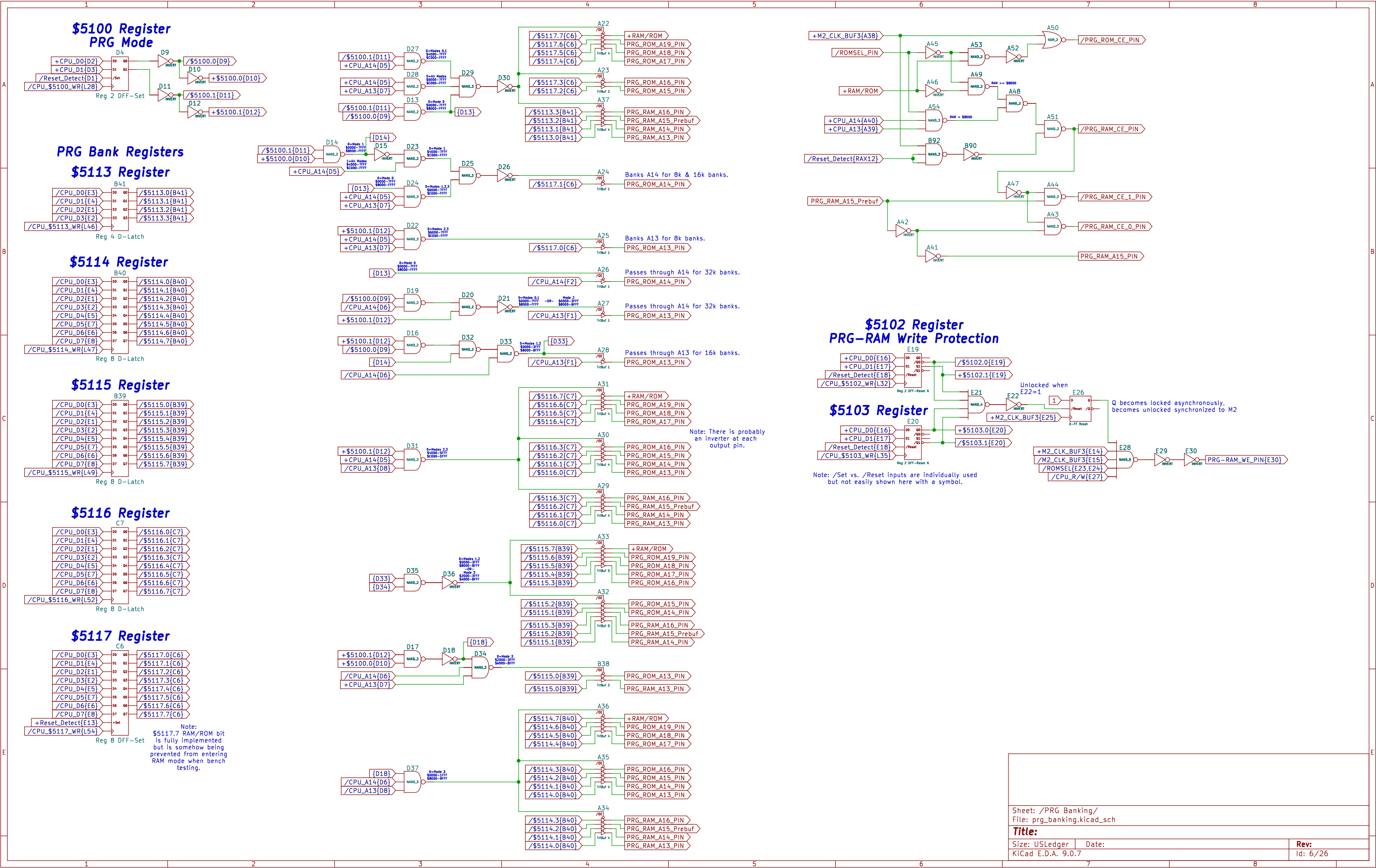
Note: /Set vs. /Reset inputs are individually used but not easily shown here with a symbol.

Note: \$5117.7 RAM/ROM bit is fully implemented but is somehow being prevented from entering RAM mode when bench testing.

Sheet: /PRG Banking/
File: prg_banking.kicad_sch

Title:

Size: USLdger Date: Rev: KiCad E.D.A. 9.0.7 Id: 6/26



\$5100 Register PRG Mode

PRG Bank Registers \$5113 Register

\$5114 Register

\$5115 Register

\$5116 Register

\$5117 Register

\$5102 Register PRG-RAM Write Protection

\$5103 Register

Note: There is probably an inverter at each output pin.

Note: /Set vs. /Reset inputs are individually used but not easily shown here with a symbol.

Note: Q becomes locked asynchronously, becomes unlocked synchronized to M2

Sheet: /PRG Banking/
File: prg_banking.kicad_sch

Title:

Size: USLdger Date: Rev: KiCad E.D.A. 9.0.7 Id: 6/26

[illegible]

\$5100 Register PRG Mode

Reg 2 DFF-Set

PRG Bank Registers

\$5113 Register

Reg 4 D-Latch

\$5114 Register

Reg 8 D-Latch

\$5115 Register

Reg 8 D-Latch

\$5116 Register

Reg 8 D-Latch

\$5117 Register

Reg 8 DFF-Set

Note: \$5117.7 RAM/ROM bit is fully implemented but is somehow being prevented from entering RAM mode when bench testing.

\$5102 Register PRG-RAM Write Protection

Reg 2 DFF-Reset N

\$5103 Register

Reg 2 DFF-Reset N

Note: /Set vs. /Reset inputs are individually used but not easily shown here with a symbol.

Note: There is probably an inverter at each output pin.

Q becomes locked asynchronously, becomes unlocked synchronized to M2

Sheet: /PRG Banking/
File: prg_banking.kicad_sch

Title:

Size: USLdger Date: Rev: KiCad E.D.A. 9.0.7 Id: 6/26

[illegible]

This diagram illustrates the PRG Banking logic for a system, showing the internal structure of several registers and the logic for PRG-RAM Write Protection.

Register Structures

\$5100 Register PRG Mode

Inputs: +CPU_D0{D2}, +CPU_D1{D3}, /Reset_Detect{D1}, /CPU_\$5100_WR{L28}

Outputs: /\$5100.0{D9}, +\$5100.0{D10}, /\$5100.1{D11}, +\$5100.1{D12}

Reg 2 DFF-Set

PRG Bank Registers

\$5113 Register

Inputs: /CPU_D0{E3}, /CPU_D1{E4}, /CPU_D2{E1}, /CPU_D3{E2}, /CPU_\$5113_WR{L46}

Outputs: /\$5113.0{B41}, /\$5113.1{B41}, /\$5113.2{B41}, /\$5113.3{B41}

Reg 4 D-Latch

\$5114 Register

Inputs: /CPU_D0{E3}, /CPU_D1{E4}, /CPU_D2{E1}, /CPU_D3{E2}, /CPU_D4{E5}, /CPU_D5{E7}, /CPU_D6{E6}, /CPU_D7{E8}, /CPU_\$5114_WR{L47}

Outputs: /\$5114.0{B40}, /\$5114.1{B40}, /\$5114.2{B40}, /\$5114.3{B40}, /\$5114.4{B40}, /\$5114.5{B40}, /\$5114.6{B40}, /\$5114.7{B40}

Reg 8 D-Latch

\$5115 Register

Inputs: /CPU_D0{E3}, /CPU_D1{E4}, /CPU_D2{E1}, /CPU_D3{E2}, /CPU_D4{E5}, /CPU_D5{E7}, /CPU_D6{E6}, /CPU_D7{E8}, /CPU_\$5115_WR{L49}

Outputs: /\$5115.0{B39}, /\$5115.1{B39}, /\$5115.2{B39}, /\$5115.3{B39}, /\$5115.4{B39}, /\$5115.5{B39}, /\$5115.6{B39}, /\$5115.7{B39}

Reg 8 D-Latch

\$5116 Register

Inputs: /CPU_D0{E3}, /CPU_D1{E4}, /CPU_D2{E1}, /CPU_D3{E2}, /CPU_D4{E5}, /CPU_D5{E7}, /CPU_D6{E6}, /CPU_D7{E8}, /CPU_\$5116_WR{L52}

Outputs: /\$5116.0{C7}, /\$5116.1{C7}, /\$5116.2{C7}, /\$5116.3{C7}, /\$5116.4{C7}, /\$5116.5{C7}, /\$5116.6{C7}, /\$5116.7{C7}

Reg 8 D-Latch

\$5117 Register

Inputs: /CPU_D0{E3}, /CPU_D1{E4}, /CPU_D2{E1}, /CPU_D3{E2}, /CPU_D4{E5}, /CPU_D5{E7}, /CPU_D6{E6}, /CPU_D7{E8}, +Reset_Detect{E13}, /CPU_\$5117_WR{L54}

Outputs: /\$5117.0{C6}, /\$5117.1{C6}, /\$5117.2{C6}, /\$5117.3{C6}, /\$5117.4{C6}, /\$5117.5{C6}, /\$5117.6{C6}, /\$5117.7{C6}

Reg 8 DFF-Set

Note: \$5117.7 RAM/ROM bit is fully implemented but is somehow being prevented from entering RAM mode when bench testing.

PRG-RAM Write Protection

\$5102 Register

Inputs: +CPU_D0{E16}, +CPU_D1{E17}, /Reset_Detect{E18}, /CPU_\$5102_WR{L32}

Outputs: /\$5102.0{E19}, +\$5102.1{E19}

Reg 2 DFF-Reset N

\$5103 Register

Inputs: +CPU_D0{E16}, +CPU_D1{E17}, /Reset_Detect{E18}, /CPU_\$5103_WR{L35}

Outputs: +\$5103.0{E20}, /\$5103.1{E20}

Reg 2 DFF-Reset N

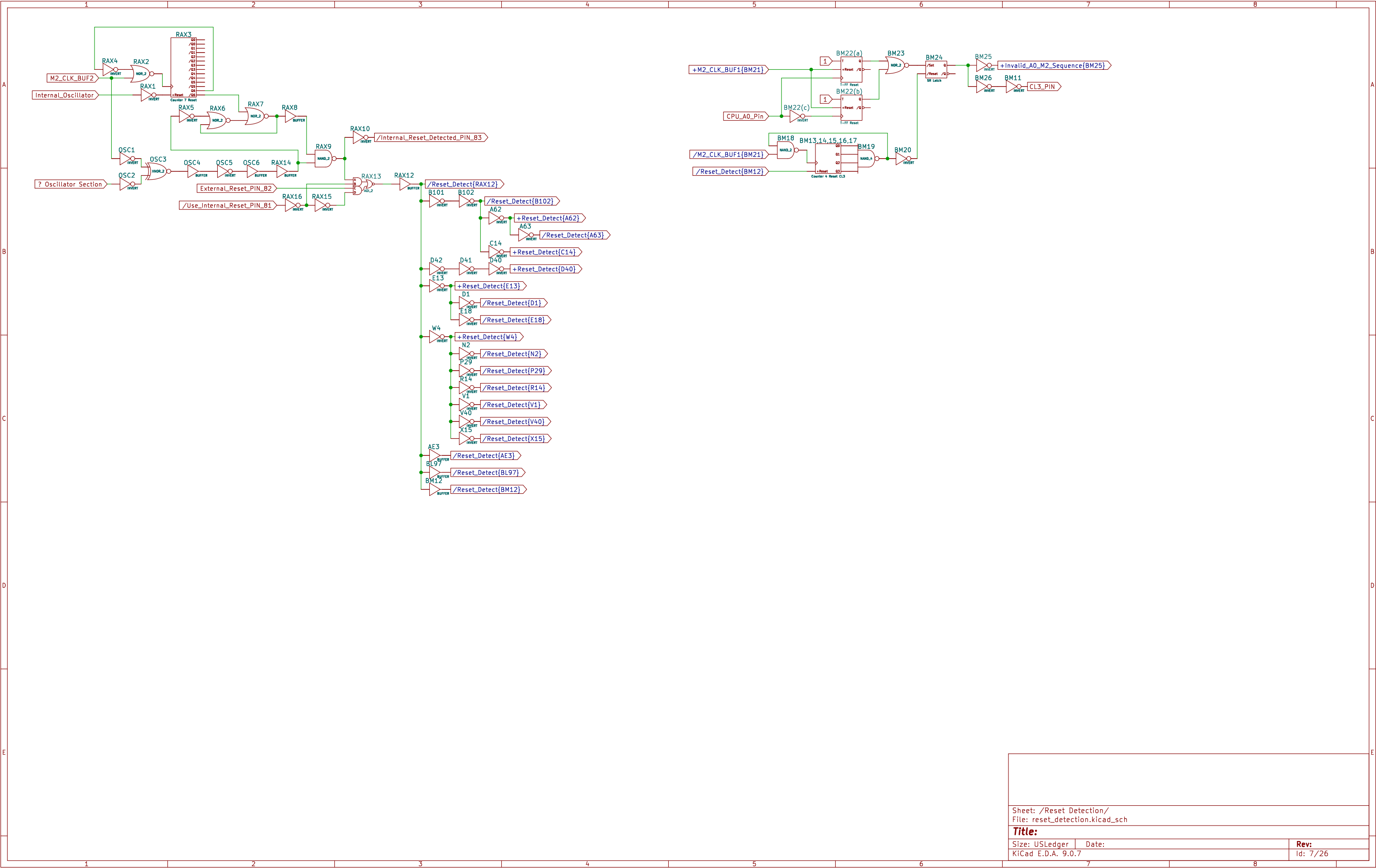
Note: /Set vs. /Reset inputs are individually used but not easily shown here with a symbol.

Logic Details

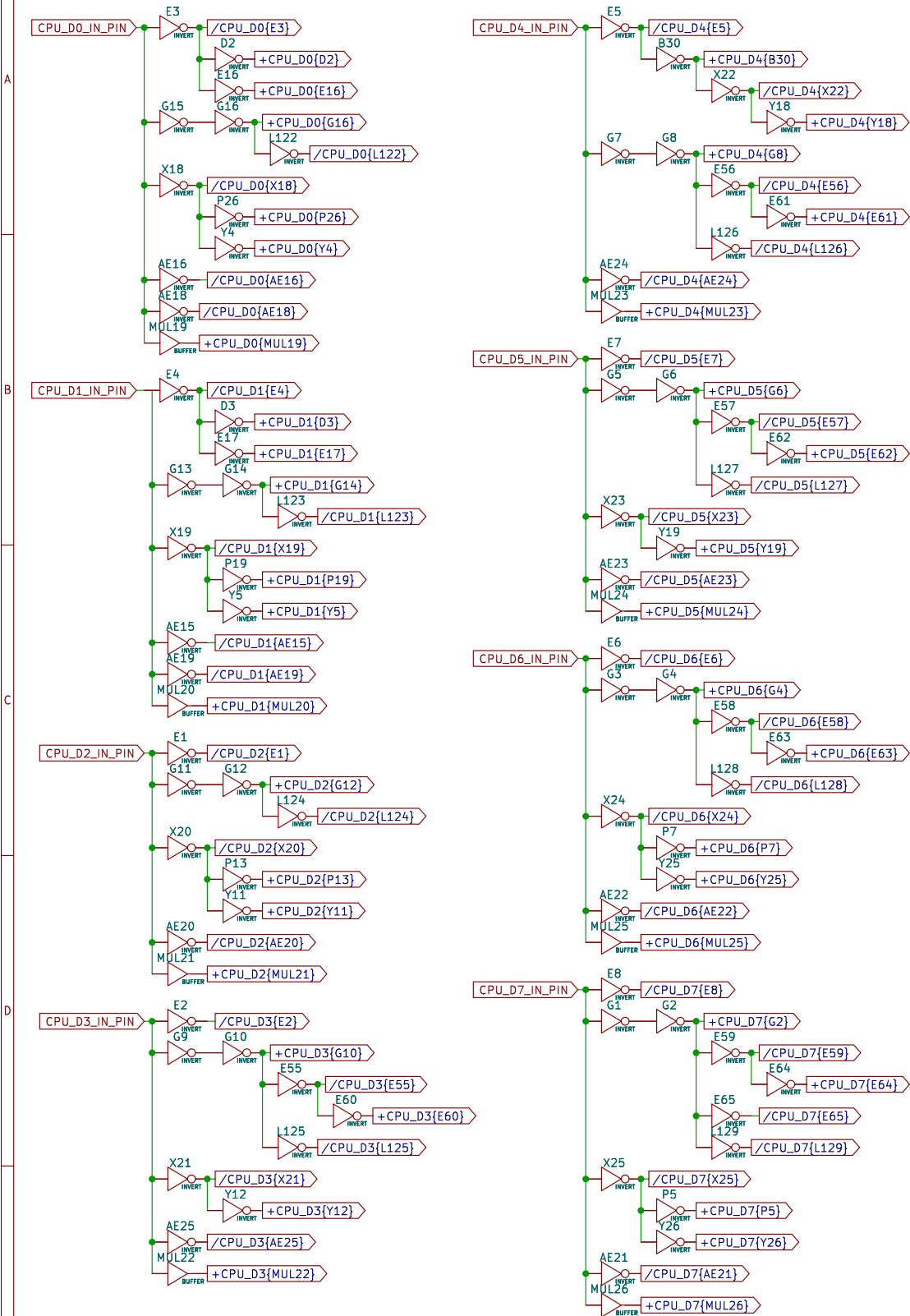
The diagram shows the internal logic for PRG-RAM Write Protection, including the PRG-RAM_CE_PIN, PRG-RAM_CE_1_PIN, PRG-RAM_CE_0_PIN, and PRG-RAM_A15_PIN signals.

Key logic components include:

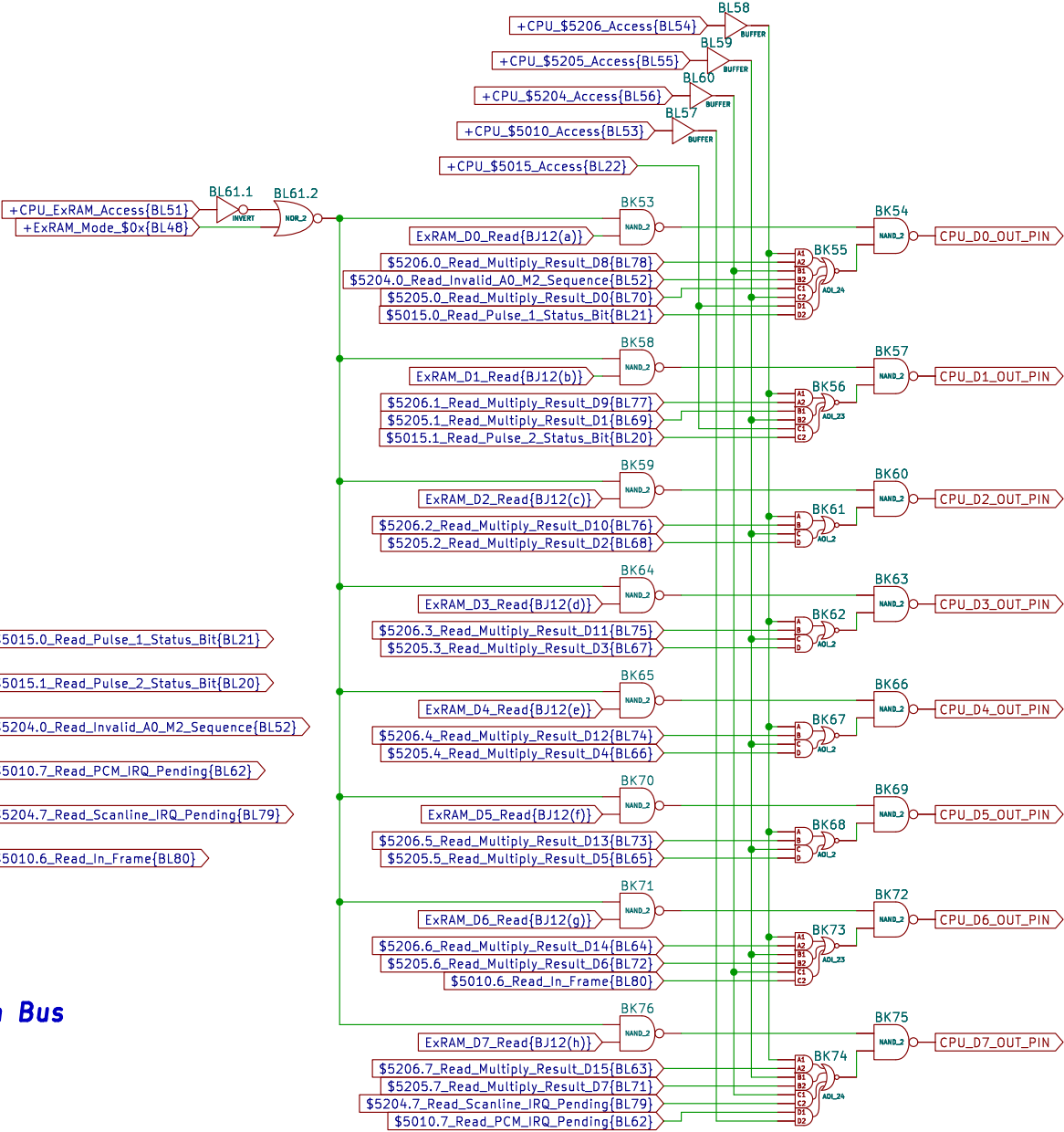
- AND gates (A22, A23, A24, A25, A26, A27, A28, A29, A30, A31, A32, A33, A34, A35, A36, A37, A42, A43, A44, A45, A46, A47, A48, A49, A50, A51, B90, B92)
- OR gates (D19, D20, D21, D22, D23, D24, D25, D26, D27, D28, D29, D30, D31, D32, D33, D34, D35, D36, D37, D38)
- NOT gates (D9, D10, D11, D12, D13, D14, D15, D16, D17, D18, D21, D22, D23, D24, D25, D26, D27, D28, D29, D30, D31, D32, D33, D34, D35, D36, D37, D38)
- TriBufs (A22, A23, A24, A25, A26, A27, A28, A29, A30, A31, A32, A33, A34, A35, A36, A37, A42, A43, A44, A45, A46, A47, A48, A49, A50, A51, B90, B92)
- RAM (RAM = \$8000)
- ROMSEL_PIN
- PRG_ROM_A19_PIN, PRG_ROM_A18_PIN, PRG_ROM_A17_PIN, PRG_ROM_A16_PIN, PRG_ROM_A15_PIN, PRG_ROM_A14_PIN, PRG_ROM_A13_PIN, PRG_ROM_A12_PIN, PRG_ROM_A11_PIN, PRG_ROM_A10_PIN, PRG_ROM_A9_PIN, PRG_ROM_A8_PIN, PRG_ROM_A7_PIN, PRG_ROM_A6_PIN, PRG_ROM_A5_PIN, PRG_ROM_A4_PIN, PRG_ROM_A3_PIN, PRG_ROM_A2_PIN, PRG_ROM_A1_PIN, PRG_ROM_A0_PIN
- PRG_RAM_A16_PIN, PRG_RAM_A15_Prebuf, PRG_RAM_A14_PIN, PRG_RAM_A13_PIN, PRG_RAM_A12_PIN, PRG_RAM_A11_PIN, PRG_RAM_A10_PIN, PRG_RAM_A9_PIN, PRG_RAM_A8_PIN, PRG_RAM_A7_PIN, PRG_RAM_A6_PIN, PRG_RAM_A5_PIN, PRG_RAM_A4_PIN, PRG_RAM_A3_PIN, PRG_RAM_A2_PIN, PRG_RAM_A1_PIN, PRG_RAM_A0_PIN



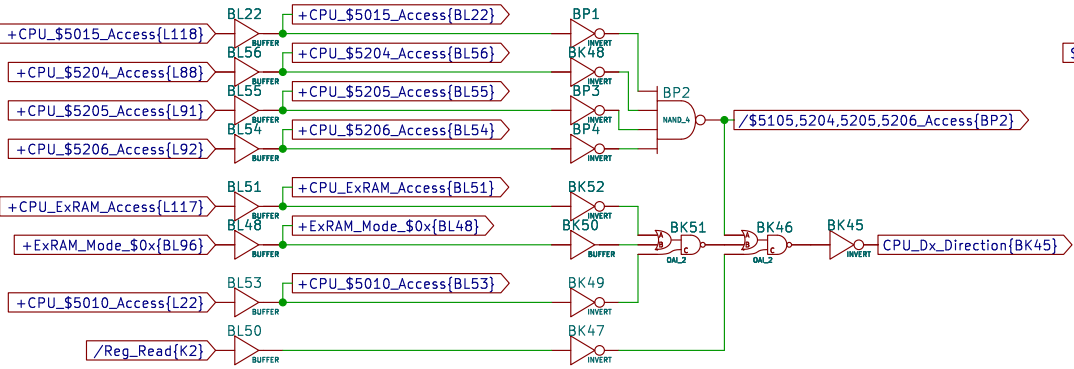
Incoming CPU Data Bus Buffering Bonanza



Multiplexing Data to be Driven onto the CPU Data Bus



Decides When to Drive the CPU Data Bus



\$5010 Register Mode & IRQ Enable

The schematic illustrates the internal logic of the \$5010 register, which is used for DAC control and IRQ enablement. It shows the data bus connections, read/write control logic, and the DAC outputs.

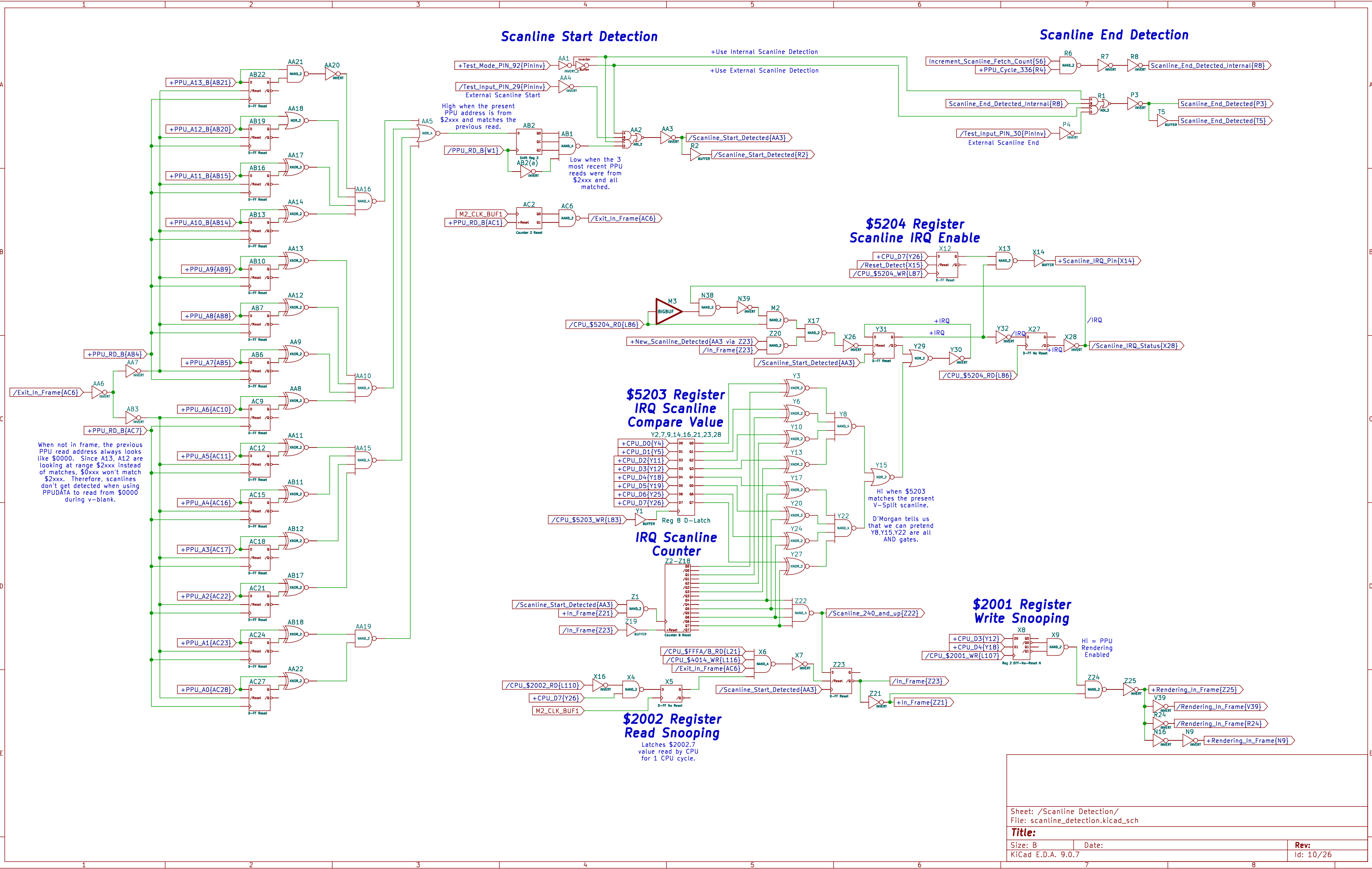
Key Components and Connections:

- Register 2 (DFF-Reset N):** Controls the DAC outputs. It is connected to the data bus via a multiplexer (K101) and a DFF (K100).
- Register 4 (D-Latch N):** Controls the DAC outputs. It is connected to the data bus via a multiplexer (K104) and a D-Latch (K105).
- DACs:** The DACs (L134, L139) are connected to the DAC outputs (PCM_DAC_Bit_0 to PCM_DAC_Bit_7).
- Control Logic:** The control logic (K106, K107, K108, K109, K110, K111, K112, K113, K114) manages the read/write operations and the IRQ enablement.

Annotations:

- Writing to \$5011 in write mode:** This section shows the logic for writing to the \$5011 register, which is used for DAC control.
- Reading from \$8000-BFFF in read mode:** This section shows the logic for reading from the \$8000-BFFF range, which is used for DAC control.
- Note:** When the value written is \$00, this loops back the existing value instead of accepting the value \$00.

Title:		
Size: USLedger	Date:	Rev:
KiCad E.D.A. 9.0.7		Id: 9/26



When not in frame, the previous PPU read address always looks like \$0000. Since A13, A12 are looking at range \$2xxx instead of matches, \$0xxx won't match \$2xxx. Therefore, scanlines don't get detected when using PPUDATA to read from \$0000 during v-blank.

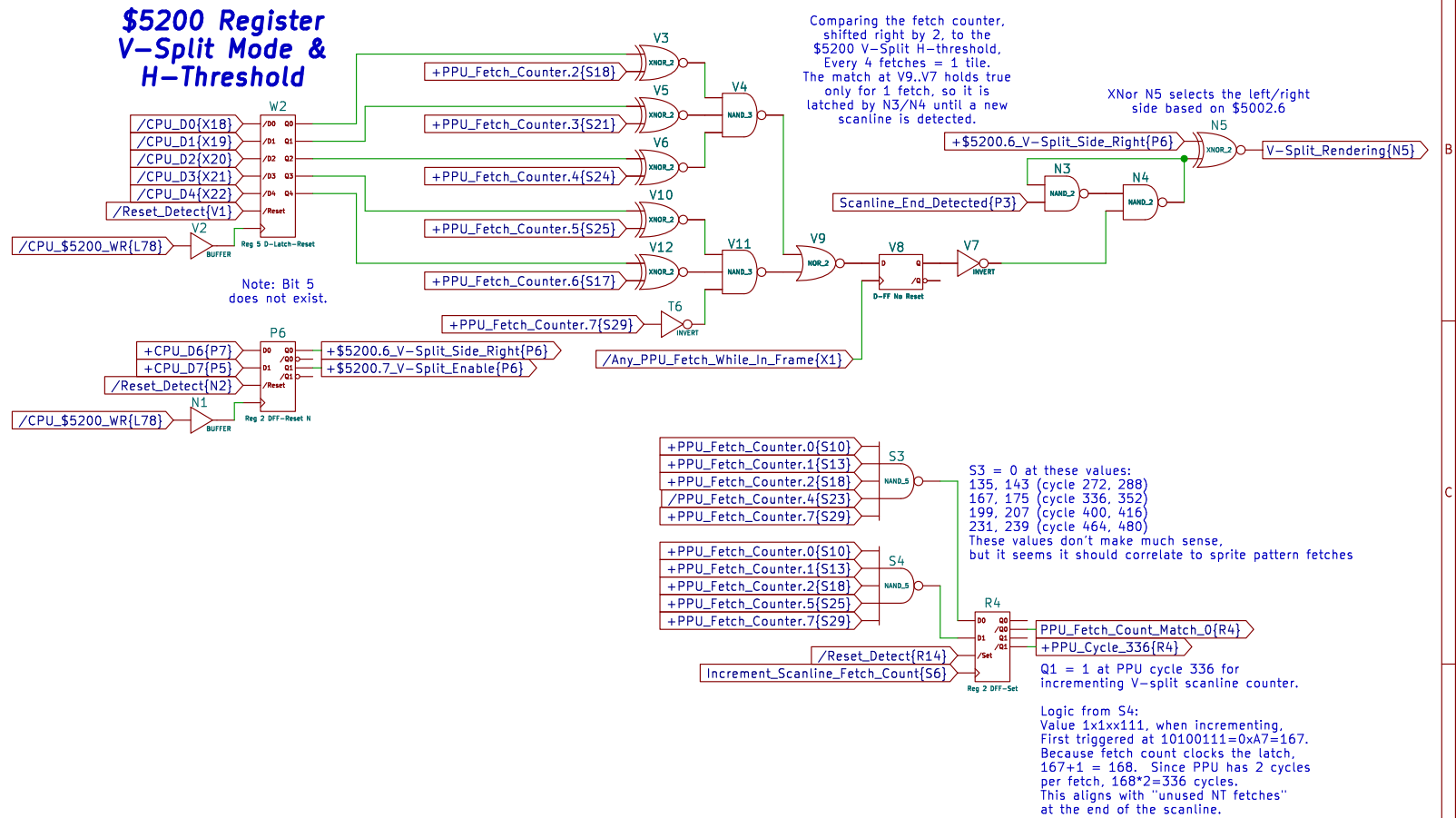
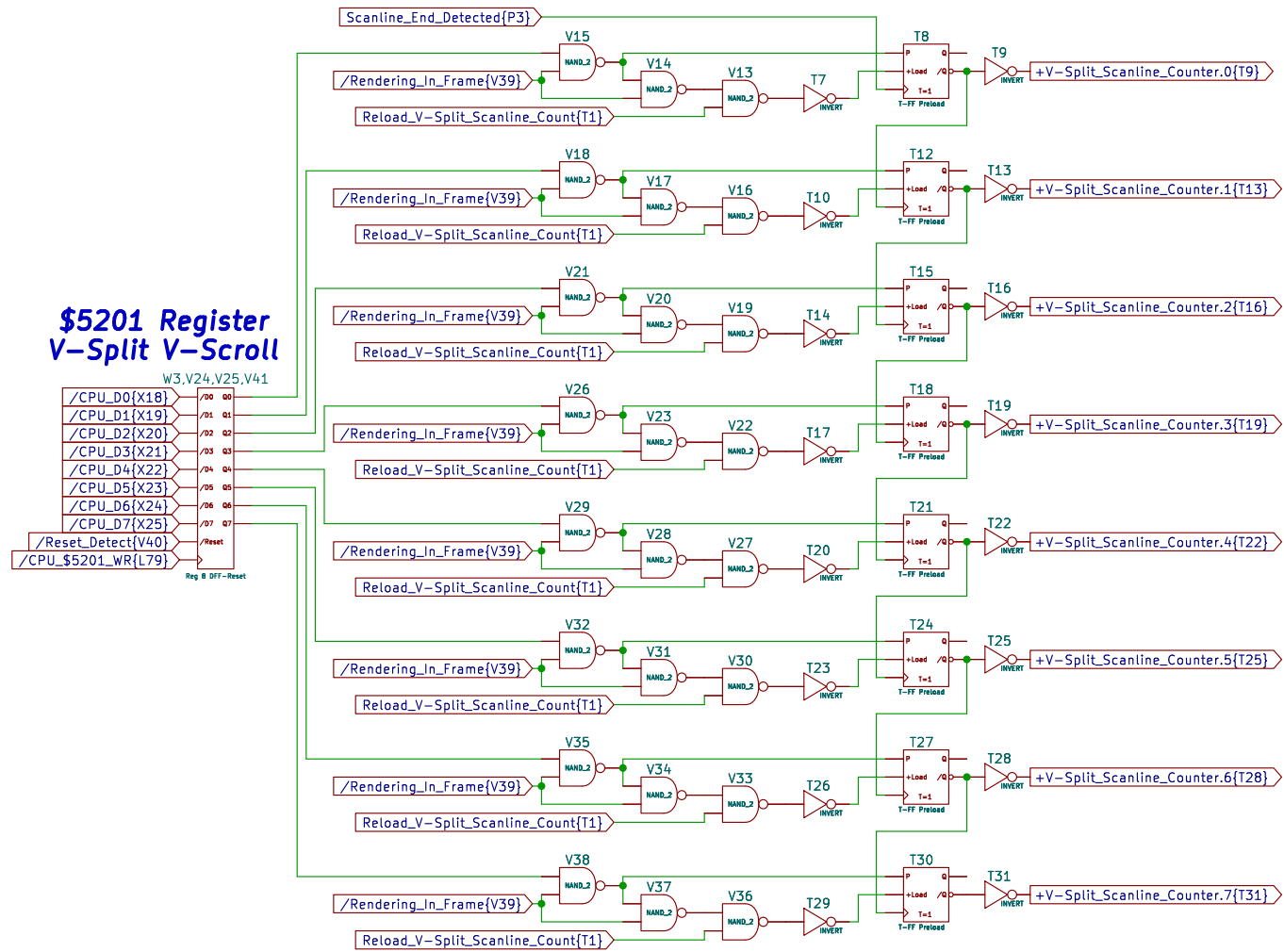
High when the present PPU address is from \$2xxx and matches the previous read.

Low when the 3 most recent PPU reads were from \$2xxx and all matched.

Hi when \$5203 matches the present V-Split scanline.

D'Morgan tells us that we can pretend Y8,Y15,Y22 are all AND gates.

Hi = PPU Rendering Enabled



1	2	3	4	5	6	7	8	
---	---	---	---	---	---	---	---	--

\$5104 Register ExRAM Mode

Consider: ExRAM NEVER contains pattern tables and that should be related to how PPU /A13 is being used here.

Question: Why should we care about PPU /A13 AT ALL when the CPU has control of ExRAM address bus?

Consider: ExRAM can still be used as a nametable when ExRAM mode is set to CPU mode.

TRUE when: ExRAM is mode \$1x, which gives CPU access but no PPU access, *OR* PPU is accessing NT or AT.

That doesn't make sense because what if you have a nametable assigned to ExRAM in \$5105? It needs ExRAM Ax coming from PPU.

Signal A9 (+ExRAM Nametable Access) is VERY closely but no obvious connection

Writing to an ExRAM nametable via PPU via \$2000

Writing to ExRAM via \$5C00-5FFF in mode %10

This is the write signal for the ExRAM itself.

Sheet: /ExRAM Mode/
File: exram_mode.kicad_sch

Title:

Size: USLdger Date: Rev:
KiCad E.D.A. 9.0.7 Id: 14/26

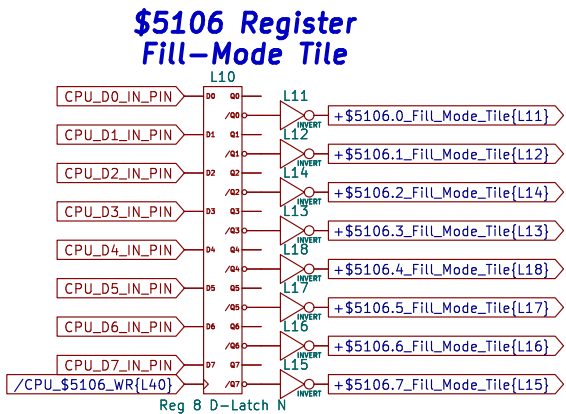
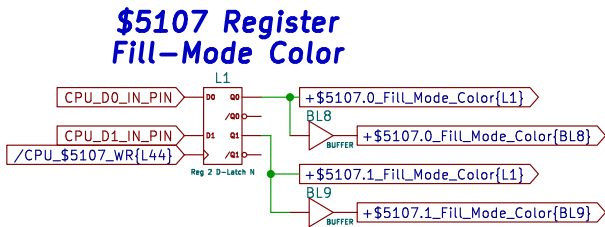
Consider:
ExRAM can still be used as a nametable
when ExRAM mode is set to CPU mode.

That doesn't make sense because
what if you have a nametable
assigned to ExRAM in \$5105?
It needs ExRAM A_x coming from P

Sheet: /ExRAM Mode/
 File: exram_mode.kicad_sch

Title:

Size: USLedge	Date:	Rev:
KiCad E.D.A. 9.0.7		Id: 14/26



Sheet: /Fill Mode/
File: fill_mode.kicad_sch

Title:

Size: USLedger

Date:

Rev:

KiCad E.D.A. 9.0.7

Id: 15/26

See spreadsheet for decoded values:
"pulse length lookup table.xlsx"

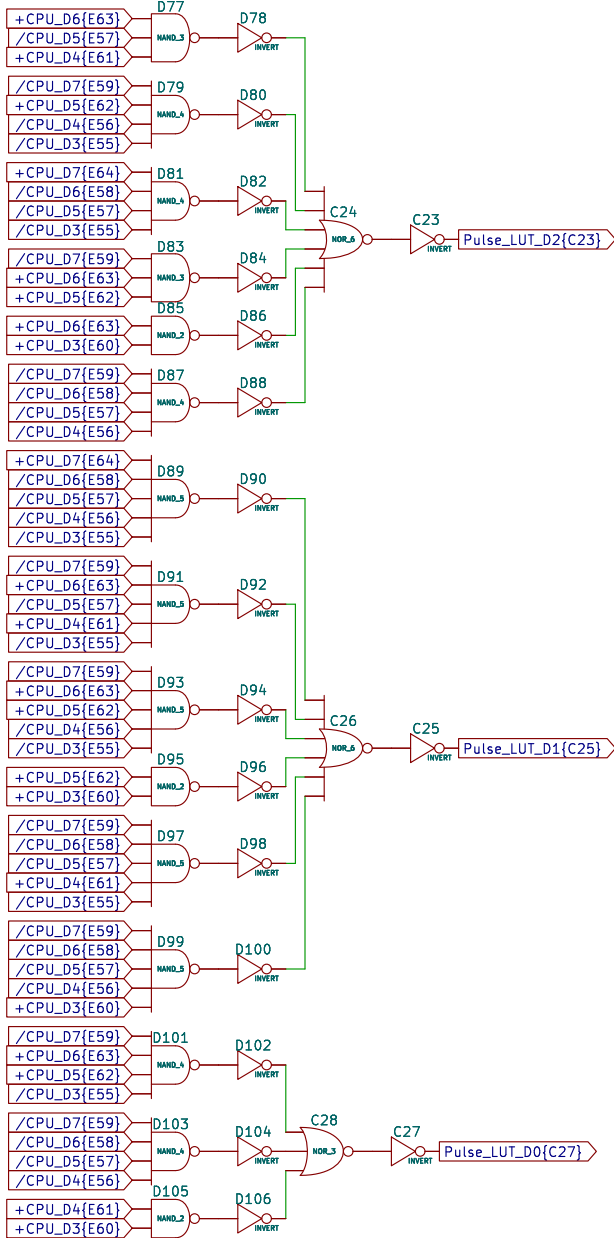
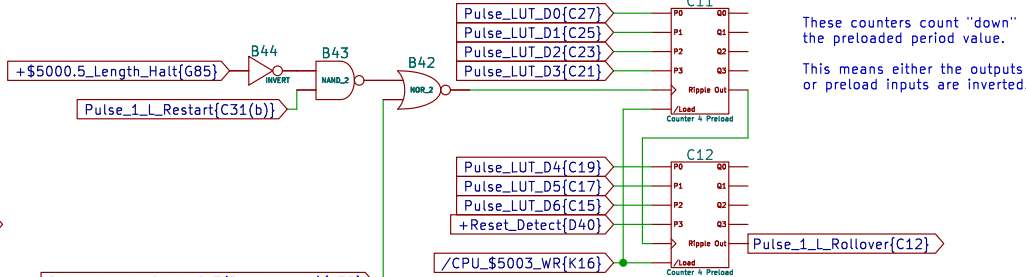


Diagram showing the connection of four LUT outputs to the P0-P3 pins of the Pinmux Out block:

- Pulse_LUT_D0{C27} connects to P0
- Pulse_LUT_D1{C25} connects to P1
- Pulse_LUT_D2{C23} connects to P2
- Pulse_LUT_D3{C21} connects to P3



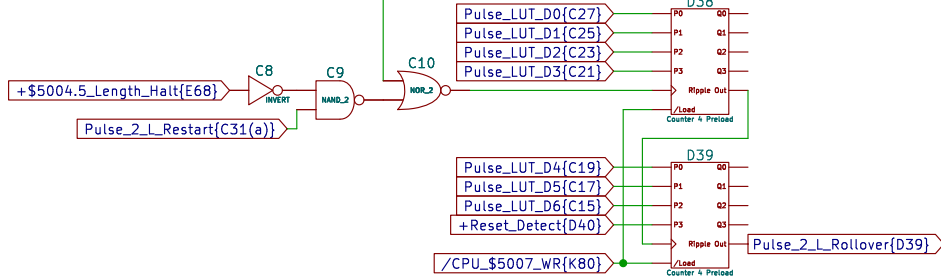
Pulse_LUT_D0{C27} → P0 Q0

Pulse_LUT_D1{C25} → P1 Q1

Pulse_LUT_D2{C23} → P2 Q2

Pulse_LUT_D3{C21} → P3 Q3

Parallel Out



1.789773 MHz
+M2_CLK_BUF2{B109}

894.886 kHz
<25% duty, depends on M2 Duty

894.886 kHz
50% duty

894.886 kHz

447.443 kHz
50% duty

3.49565 kHz
50% duty

3712

241.08 Hz
Duty seems thin

241.08 Hz or
Pin 98 Input

0=Reset Held By A67.Q4 or Pin 30

The reset inputs of ALL these counters seem to be /Reset. Hard to see why this acts different than T-FF counters elsewhere.

A65 resets Q7 of A66.

Important: A63 resets Q4 of A67.

A66.2 in this logic can only create a phase shift and not affect the period since it doesn't cause A66 to reset.

All of these outputs are 241.08 Hz -or- Pin 98 Input

All of these outputs are
241.08 Hz
-or-
Pin 98 Input

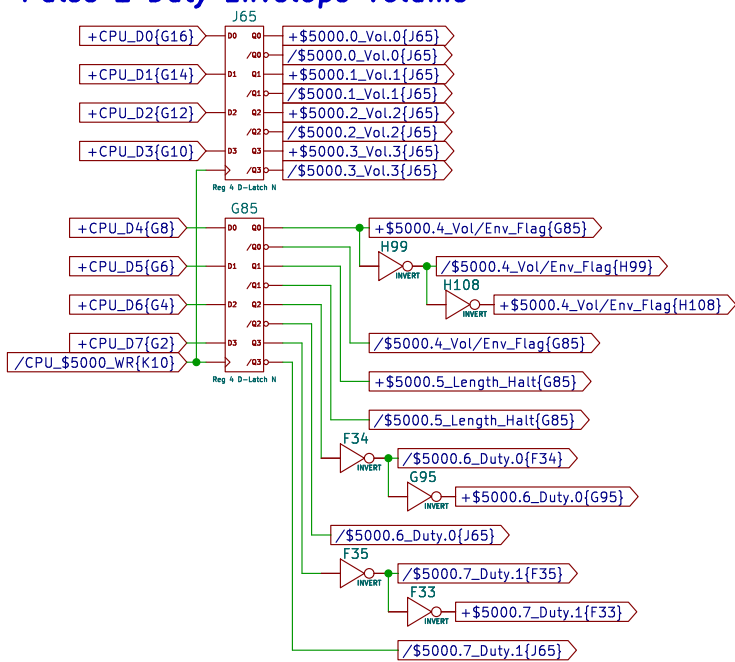
Sheet: /Pulse Length Lookup Table/
File: pulse_length_lookup_table.kicad_sch

Title:

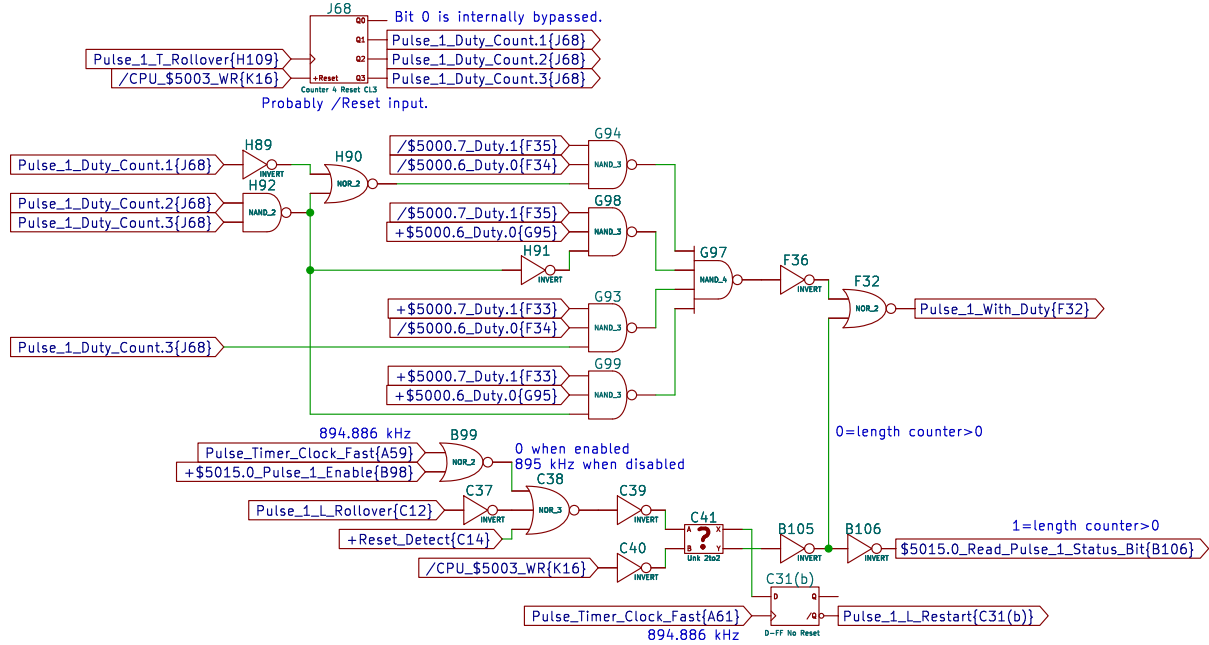
Size: USLedge	Date:
KiCad E.D.A. 9.0.7	

Rev:
Id: 16/26

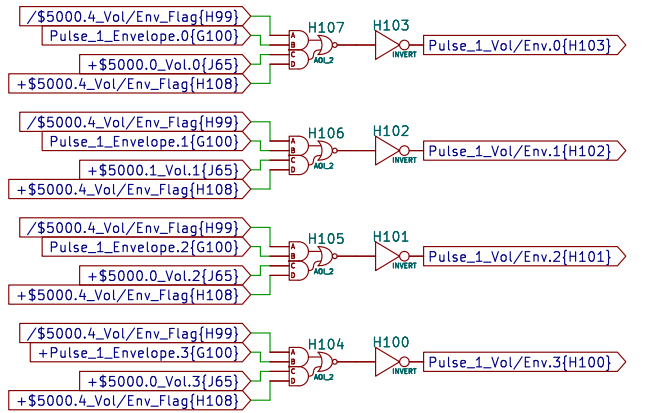
\$5000 Register Pulse 1 Duty Envelope Volume



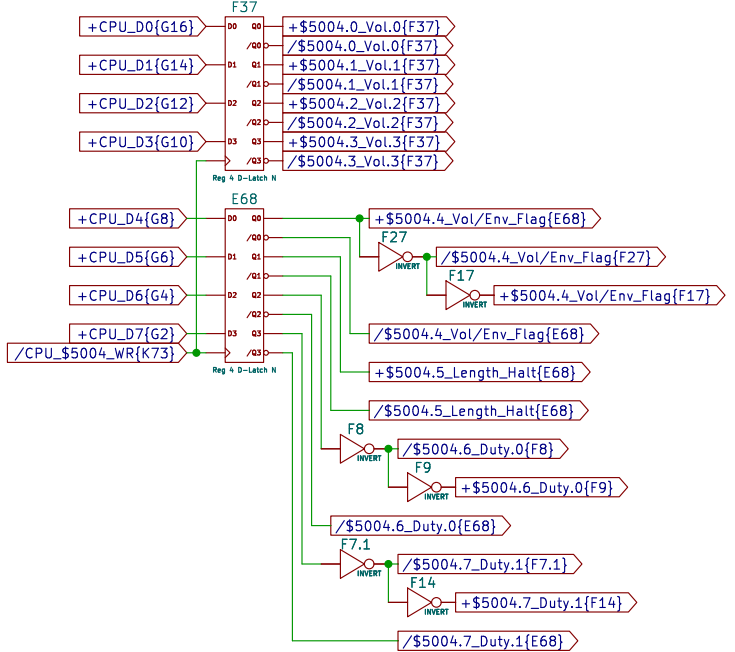
Pulse 1 Duty Cycle Counter



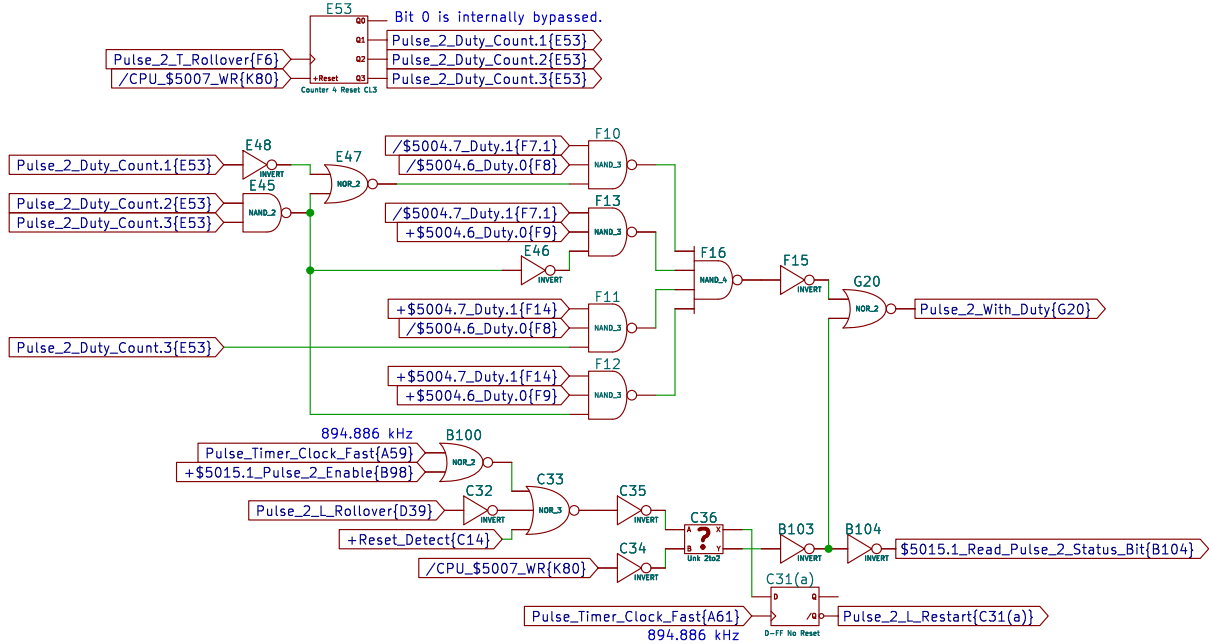
Pulse 1 Envelope/Volume Multiplexing



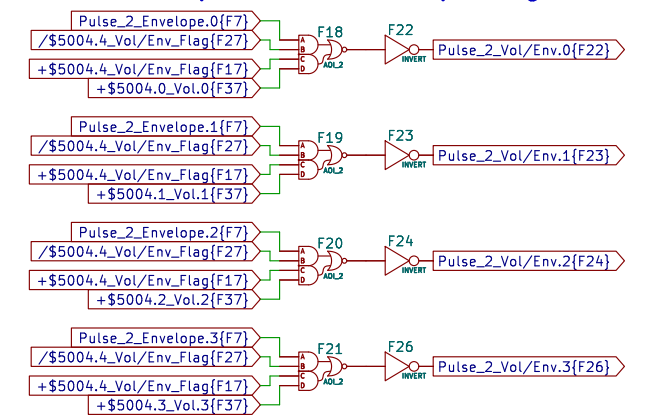
\$5004 Register Pulse 2 Duty Envelope Volume



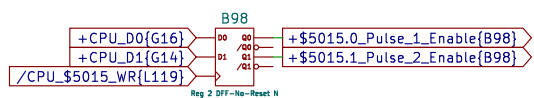
Pulse 2 Duty Cycle Counter



Pulse 2 Envelope/Volume Multiplexing



\$5015 Write Register Pulse 1&2 Status

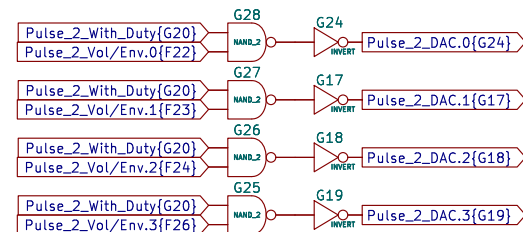
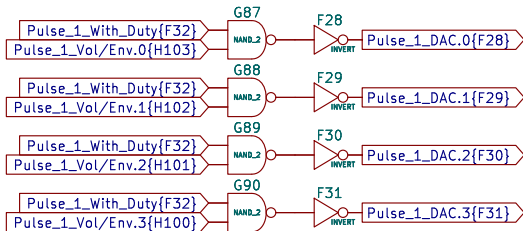


Sheet: /Pulse Duty Envelope Volume/
File: pulse_duty_envelope_volume.kicad_sch

Title:

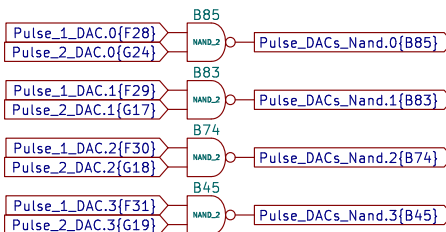
Size: USLdger Date:
KiCad E.D.A. 9.0.7

Rev:
Id: 18/26



```

graph LR
    P1_0[Pulse_1_DAC.0[F28]] --- NOR0[NOR.2]
    P2_0[Pulse_2_DAC.0[G24]] --- NOR0
    NOR0 --- B84[Pulse_DACs_Nor.0[B84]]
    
    P1_1[Pulse_1_DAC.1[F29]] --- NOR1[NOR.2]
    P2_1[Pulse_2_DAC.1[G17]] --- NOR1
    NOR1 --- B82[Pulse_DACs_Nor.1[B82]]
    
    P1_2[Pulse_1_DAC.2[F30]] --- NOR2[NOR.2]
    P2_2[Pulse_2_DAC.2[G18]] --- NOR2
    NOR2 --- B73[Pulse_DACs_Nor.2[B73]]
    
    P1_3[Pulse_1_DAC.3[F31]] --- NOR3[NOR.2]
    P2_3[Pulse_2_DAC.3[G19]] --- NOR3
    NOR3 --- B53[Pulse_DACs_Nor.3[B53]]
  
```



Note: The adder's logic is verified correct in Pulse DAC Adder Logic.xlsx

This logic amounts to XOR

Pulse_DACs_Nand.0{B85}

Pulse_DACs_Nor.0{B84}

B50 INVERT

B49 NAND.2

B51 INVERT

Pulse_DACs_Nand.1{B83}

Pulse_DACs_Nor.1{B82}

B78 INVERT

B79 INVERT

B88 INVERT

B89 INVERT

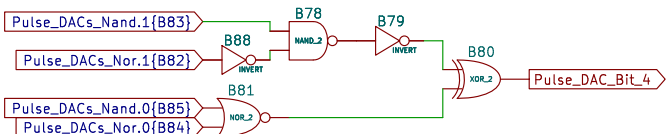
B80 XOR.2

Pulse_DAC_Bit_1

Pulse_DAC_Bit_2

Pulse_DAC_Bit_3

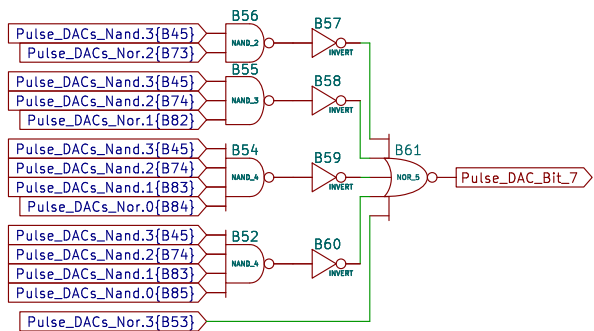
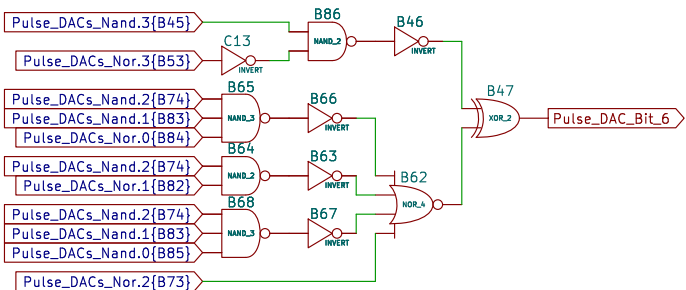
Pulse_DAC_Bit_4

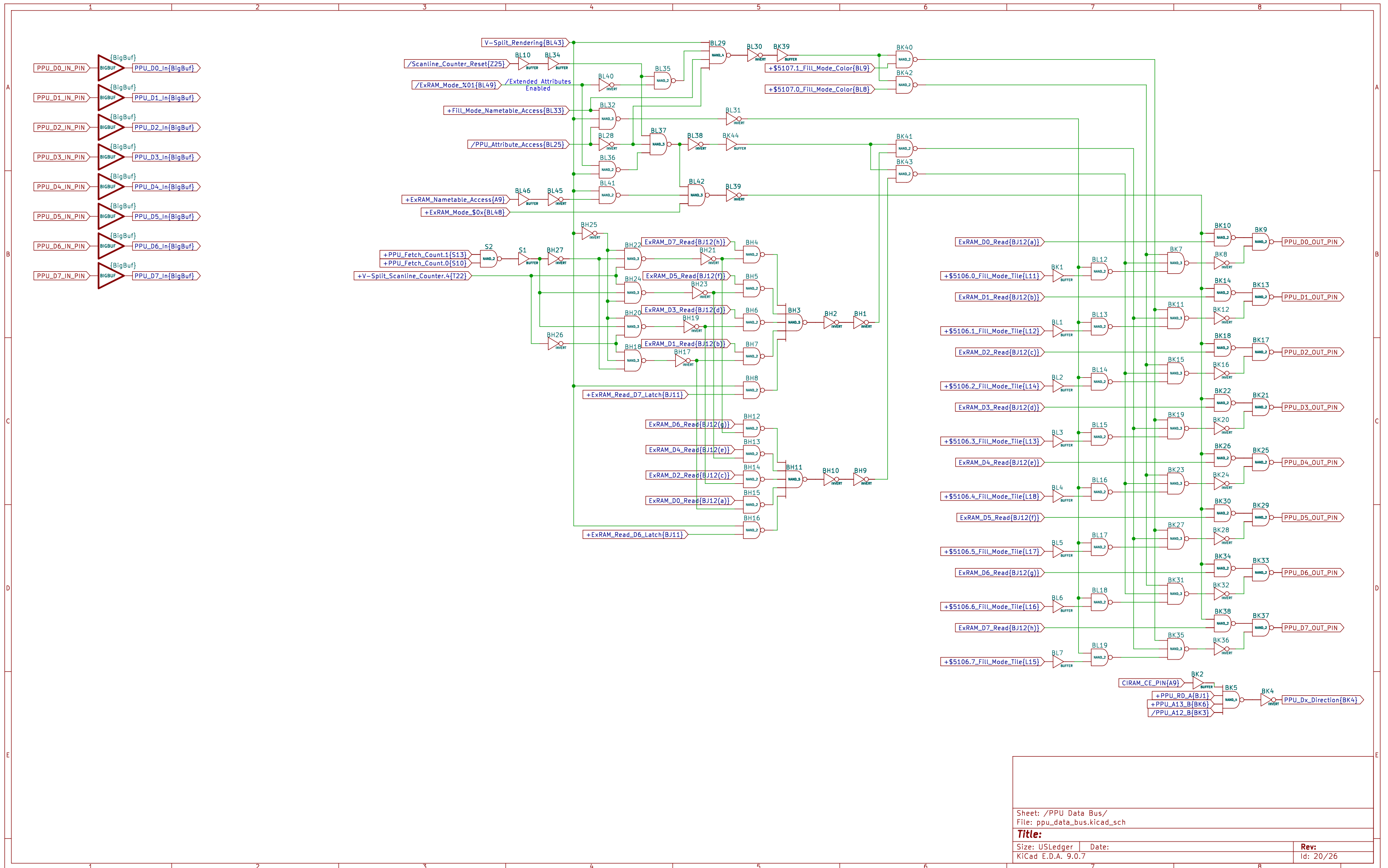


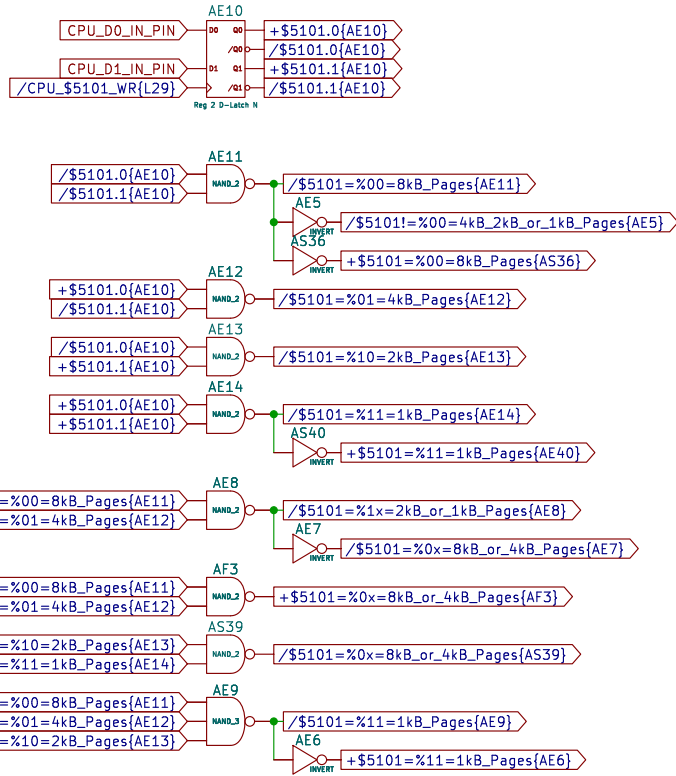
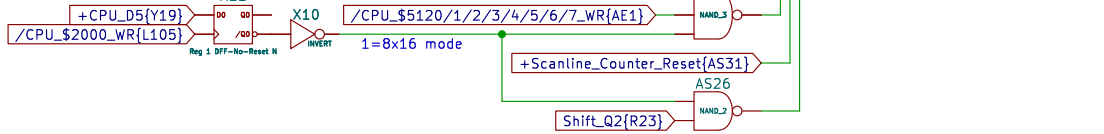
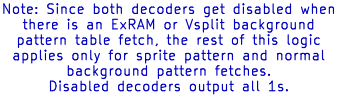
Lowest 3 bits
are not used.

This

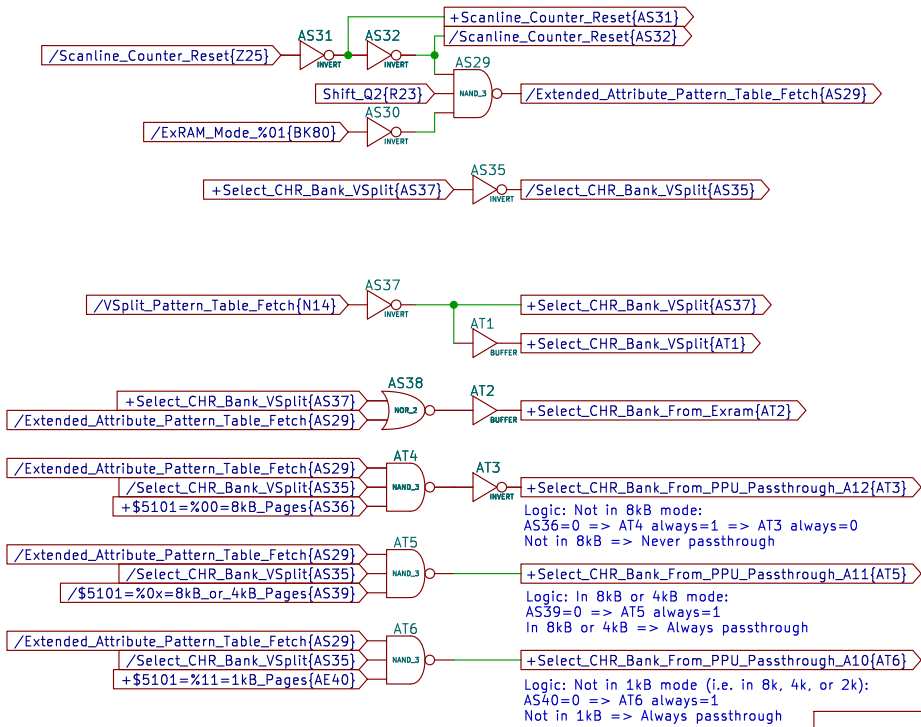
B80,B72,B47:
Though they look like
XNOR gates from
elsewhere, logically
these must be XOR.



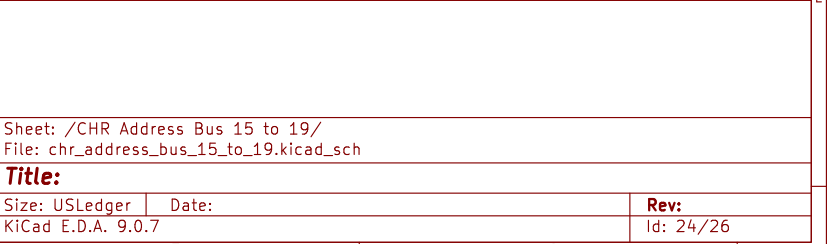




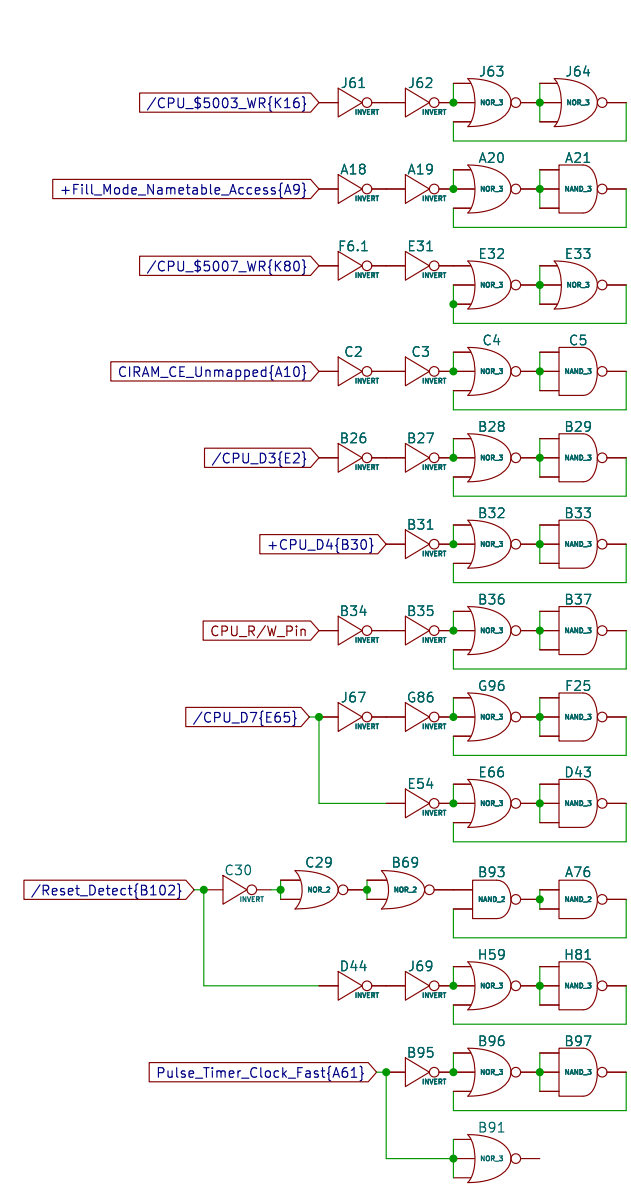
The logic in this area is
strangely unoptimized.



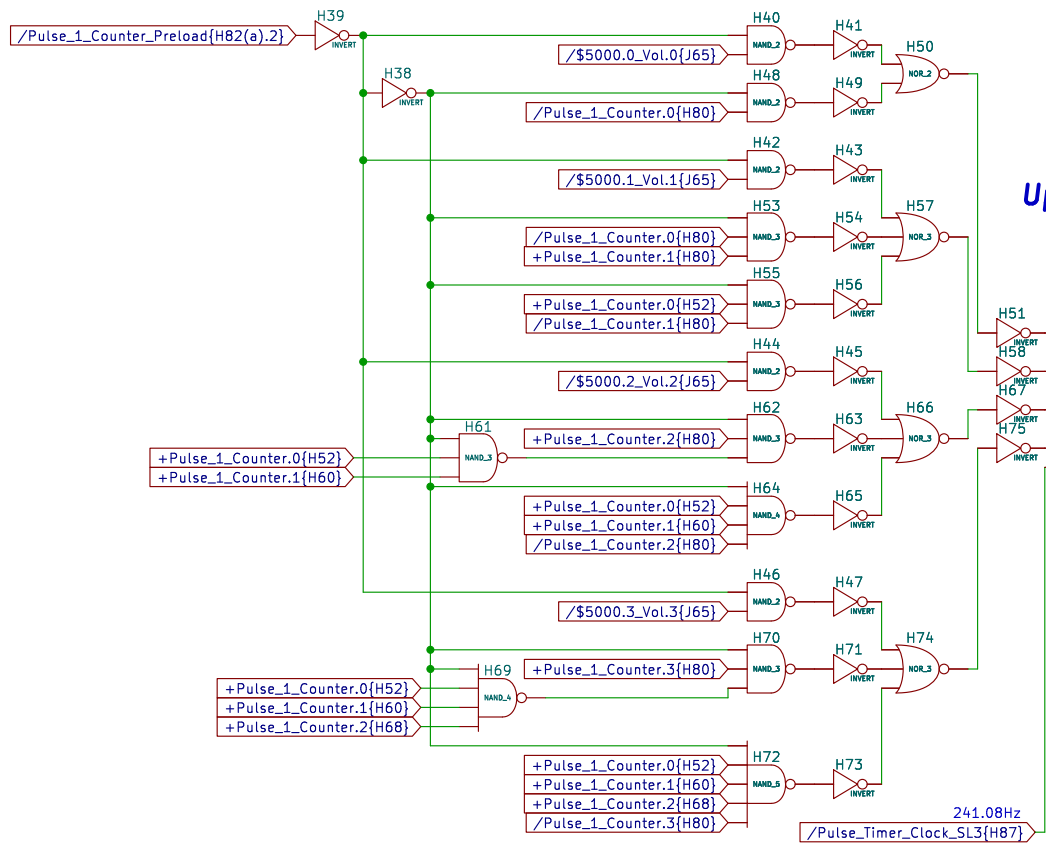
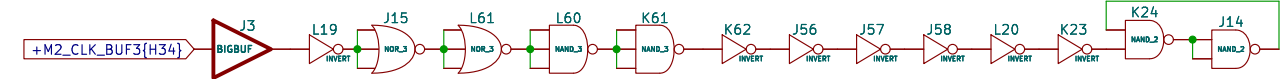
Size: USLedger	Date:	Rev:
KiCad E.D.A. 9.0.7		Id: 22/26



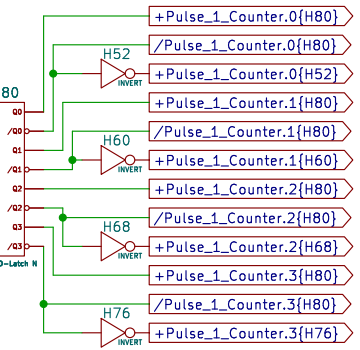
This sheet contains all residual items that don't serve any apparent purpose.



M2 Spaghetti That Goes Nowhere

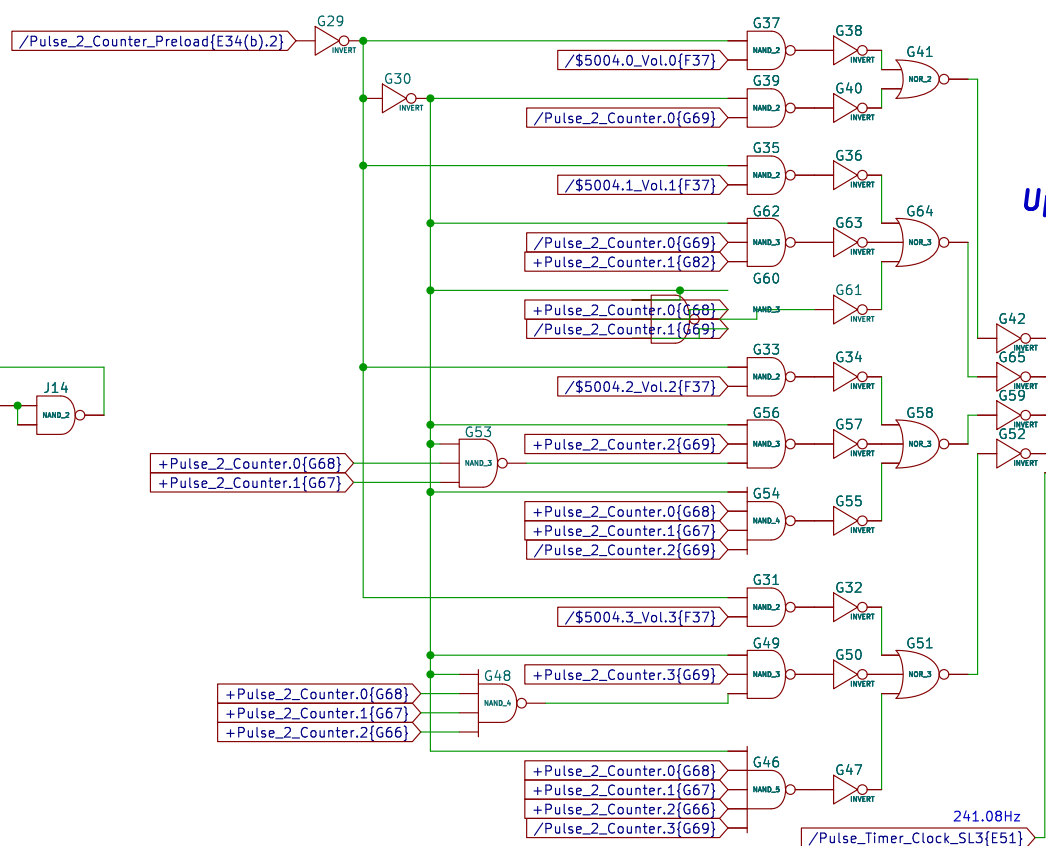
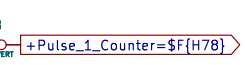


Pulse 1
Unused 4-bit Binary
Up-Counter With Preload



The only output that goes elsewhere is H78, which gets latched by H82(a), coming directly back here as /Pulse_1_Counter_Preload; all self-contained. It does not appear to provide any other external signals, so I don't think it actually does anything. It seems like an abandoned envelope counter that was left in.

It latches the 4-bit pulse volume, then counts up at a rate of 240 Hz. Nothing stops it from counting or rolling over from \$F back to \$0.



Pulse 2
Unused 4-bit Binary
Up-Counter With Preload

